Lab 13

Working with Rate Limiting Policies

WSO2 API Manager 4.7.0 · Practical exercise

Before You Begin

URLs in this guide use placeholders. Replace them with the values of the API Manager you are using:

- **`<APIM_HOST>`** – host name or IP address of the API Manager server (localhost when API Manager runs on your own laptop).
- **`<HTTPS_PORT>`** – API Manager HTTPS port: 9443 plus any port offset (Publisher, Developer Portal, Admin Portal, Management Console, token endpoint).
- **`<GW_HTTPS_PORT>` / `<GW_HTTP_PORT>`** – API Gateway ports: 8243 (HTTPS) and 8280 (HTTP) plus any port offset.
- **`<APIM_HOME>`** – the folder where API Manager is extracted.

The servers use a self-signed certificate by default, so the browser shows a security warning the first time: choose Advanced and continue.

Training Objective

Learn how to add new rate limiting policies and define extra properties for them using the Admin Portal. Rate limiting allows you to limit the number of hits to an API during a given period of time.

Business Scenario

PizzaShack's API is getting popular, so they want a premium Platinum business plan that allows 25 requests per minute per subscription, with bursts of at most 5 requests per second. They also want to limit a bot (Googlebot) separately, limit each user to a few calls per minute on the PizzaShack API, and block an application called Pizzaman that is misusing the API.

High-Level Steps

- Add a rate limiting policy
- Add conditions to an advanced rate limiting policy
- Add a custom rule
- Block all calls from an application by denying the application

Part 1 – Add a Rate Limiting Policy

1. Log in to the Admin Portal as **admin** (default user name and password: **admin** / **admin**):

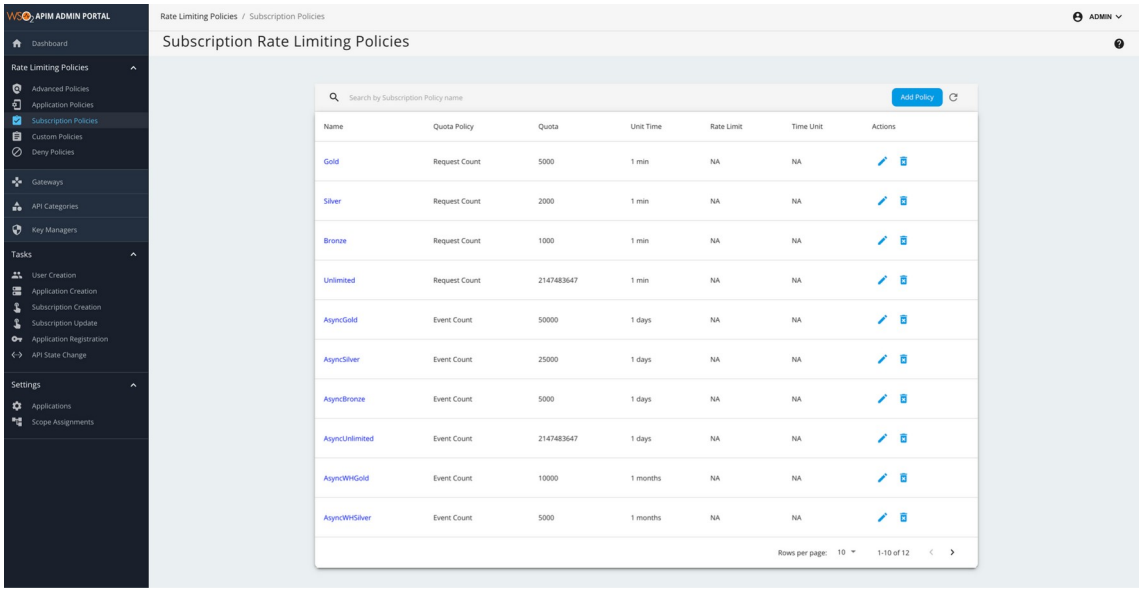
```
https://<APIM_HOST>:<HTTPS_PORT>/admin
```

Example:

```
https://localhost:9443/admin
```

CC BY 4.0

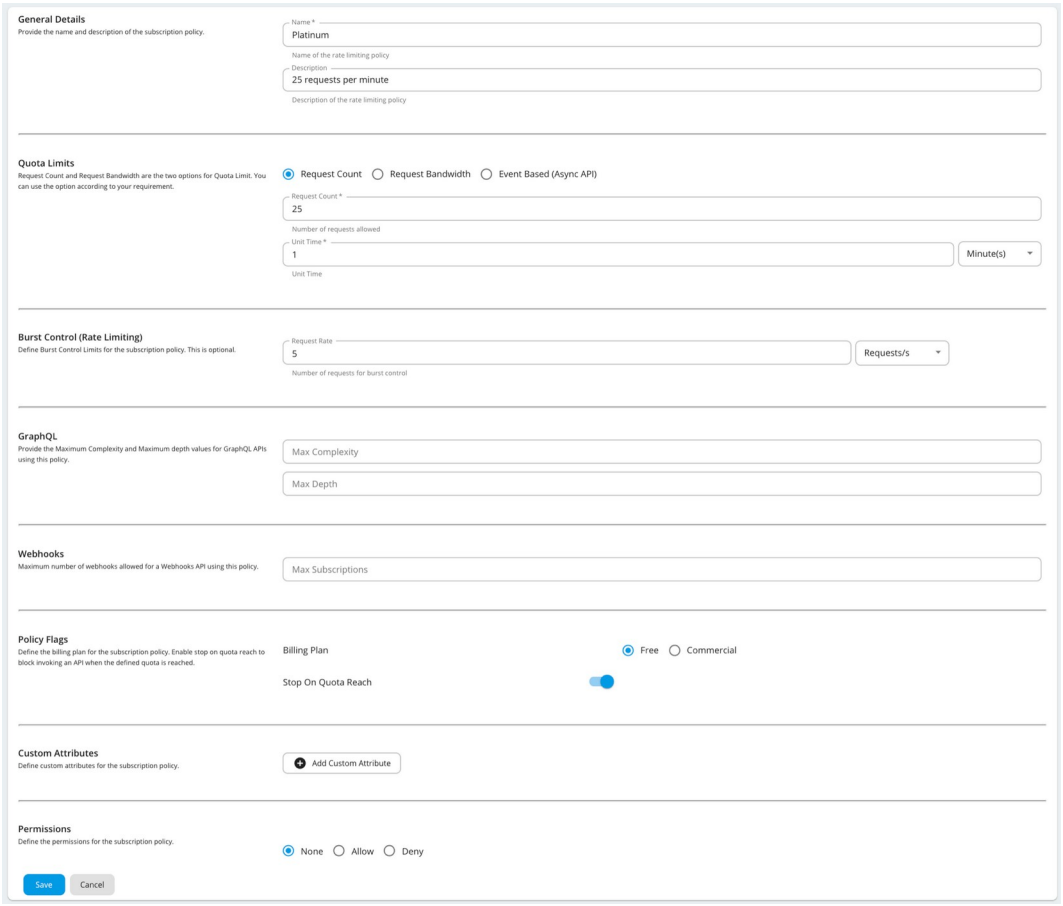
2. Under **Rate Limiting Policies** in the left menu, select **Subscription Policies** and click **Add Policy**.



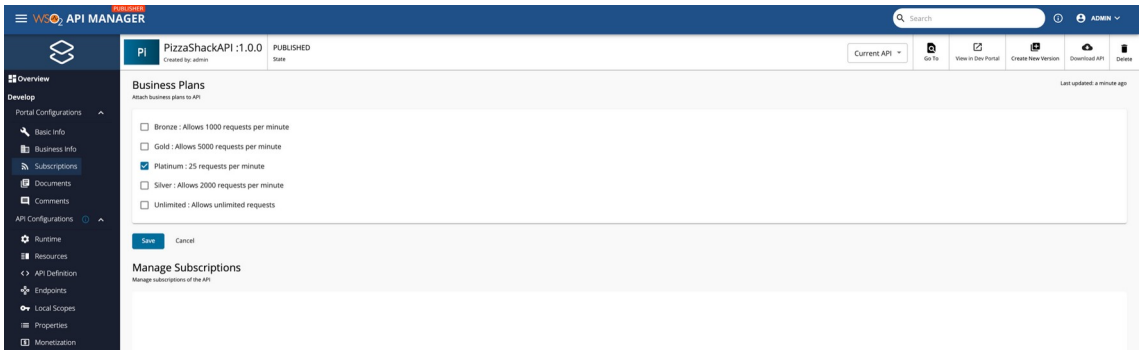
3. Enter the following information and click **Save**:

Field	Value
Name	`Platinum`
Description	25 requests per minute
Quota Limits	Request Count: 25 per 1 Minute(s)
Burst Control (Rate Limiting)	5 Requests/s
Billing Plan	Free
Stop On Quota Reach	On

CC BY 4.0



4. In the Publisher, open the **PizzaShack** API, go to **Portal Configurations → Subscriptions**, select **Platinum** under **Business Plans** (keep the plans already selected) and click **Save**. If the Publisher asks you to deploy, deploy a new revision. Platinum is now offered when someone subscribes to the PizzaShack API in the Developer Portal.



See how Platinum appears to subscribers, then test it:

5. Log in to the Developer Portal as **admin** and open the **PizzaShack** API:

```
https://<APIM_HOST>:<HTTPS_PORT>/devportal
```

Example:

```
https://localhost:9443/devportal
```

CC BY 4.0

The new plan shows in two places:

Where	What you see
Overview page → Business Plans	A card for each plan the API offers. The new card shows Platinum, 25 and Requests/min, next to the plans that were already selected (for example Gold, 5000 Requests/min).
Subscriptions (left menu) → Business Plan list	Platinum is listed together with the other plans selected in step 4.

The card shows only the quota (25 Requests/min). The burst limit (5 requests per second) is not shown, but the gateway applies it.

6. Create a test application: go to **Applications** and click **Add New Application**. Name it **PlatinumTestApp** and set **Shared Quota** to **Unlimited**, then click **Save**.

   Important: the form selects **10PerMin** (the first application policy in the list) by default. This application-level limit is checked before the subscription limit, so with 10PerMin the calls in the next step are blocked after 10 with code `900803`, before Platinum is reached.

7. Subscribe with the Platinum plan: open the PizzaShack API, click **Subscriptions**, select **PlatinumTestApp** and the **Platinum** business plan, and click **Subscribe**. Open the application, go to **Production Keys**, click **Generate Keys** and copy the access token.

8. Test the quota: send 30 requests to `GET /menu`, two per second (slow enough to stay under the burst limit). Replace `<ACCESS_TOKEN>` with the token from step 7.

   macOS / Linux

```
for i in $(seq 1 30); do
  curl -k -s -o /dev/null -w "$i %{http_code}\n" \
    -H "Authorization: Bearer <ACCESS_TOKEN>" \
    "https://<APIM_HOST>:<GW_HTTPS_PORT>/pizzashack/1.0.0/menu"
  sleep 0.5
done
```

   Example – macOS / Linux:

```
for i in $(seq 1 30); do
  curl -k -s -o /dev/null -w "$i %{http_code}\n" \
    -H "Authorization: Bearer eyJ4NXQiOiJNell4.eyJzdWIiOiJhZG1pbiJ9.ZXhhbXBsZQ" \
    "https://localhost:8243/pizzashack/1.0.0/menu"
  sleep 0.5
done
```

   Windows (Command Prompt)

```
for /L %i in (1,1,30) do @(curl -k -s -o NUL -w "%i %{http_code}\n" ^
  -H "Authorization: Bearer <ACCESS_TOKEN>" ^
  "https://<APIM_HOST>:<GW_HTTPS_PORT>/pizzashack/1.0.0/menu" ^
  & timeout /t 1 /nobreak >NUL)
```

   Example – Windows (Command Prompt):

```
for /L %i in (1,1,30) do @(curl -k -s -o NUL -w "%i %{http_code}\n" ^
  -H "Authorization: Bearer eyJ4NXQiOiJNell4.eyJzdWIiOiJhZG1pbiJ9.ZXhhbXBsZQ" ^
  "https://localhost:8243/pizzashack/1.0.0/menu" ^
  & timeout /t 1 /nobreak >NUL)
```

CC BY 4.0

Expected result (request number and HTTP status):

```
1 200
2 200
...
25 200
26 200
27 429
28 429
29 429
30 429
```

Then call the API once more to see the error that the gateway returns:

```
curl -k -H "Authorization: Bearer <ACCESS_TOKEN>" \
  "https://<APIM_HOST>:<GW_HTTPS_PORT>/pizzashack/1.0.0/menu"
```

Example:

```
curl -k -H "Authorization: Bearer eyJ4NXQiOiJNell4.eyJzdWIiOiJhZG1pbiJ9.ZXhhbXBsZQ" \
  "https://localhost:8243/pizzashack/1.0.0/menu"
```

Response (HTTP 429):

```
{"code":"900804","message":"Message throttled out",
 "description":"You have exceeded your quota .You can access API after
  2026-Nov-03 11:37:00+0000 UTC","nextAccessTime":"2026-Nov-03 11:37:00+0000 UTC"}
```

Code `900804` means the subscription quota (Platinum, 25 per minute) is used up. One or two calls above the limit can still pass, because the gateway sends its counts to the Traffic Manager and waits for its decision. From the time in `nextAccessTime`, calls work again. The count restarts every minute: if you see no 429, the loop crossed into a new minute, so run it again. In all the tests in this lab, `000` instead of a status code means the request did not reach the gateway: check the URL (only the gateway port, for example 8243).

9. Optional – test burst control: send 10 requests at the same time (macOS / Linux, or Git Bash on Windows).

macOS / Linux

```
for i in $(seq 1 10); do
  curl -k -s -o /dev/null -w "%{http_code}\n" \
    -H "Authorization: Bearer <ACCESS_TOKEN>" \
    "https://<APIM_HOST>:<GW_HTTPS_PORT>/pizzashack/1.0.0/menu" &
done; wait
```

Example – macOS / Linux:

```
for i in $(seq 1 10); do
  curl -k -s -o /dev/null -w "%{http_code}\n" \
    -H "Authorization: Bearer eyJ4NXQiOiJNell4.eyJzdWIiOiJhZG1pbiJ9.ZXhhbXBsZQ" \
    "https://localhost:8243/pizzashack/1.0.0/menu" &
done; wait
```

Some of the calls return **429** with code `900807` (more than 5 requests in one second). How many varies from run to run, because burst control is counted on each gateway node, per second.

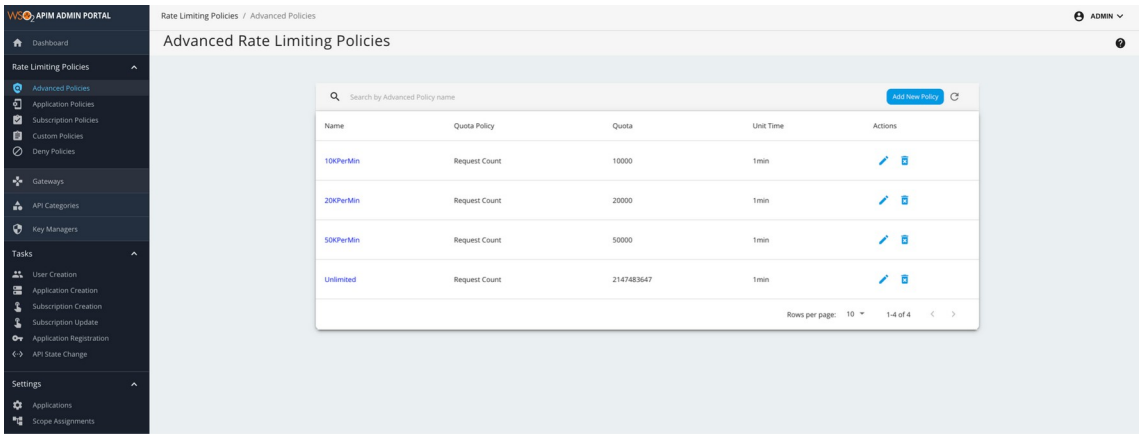
CC BY 4.0

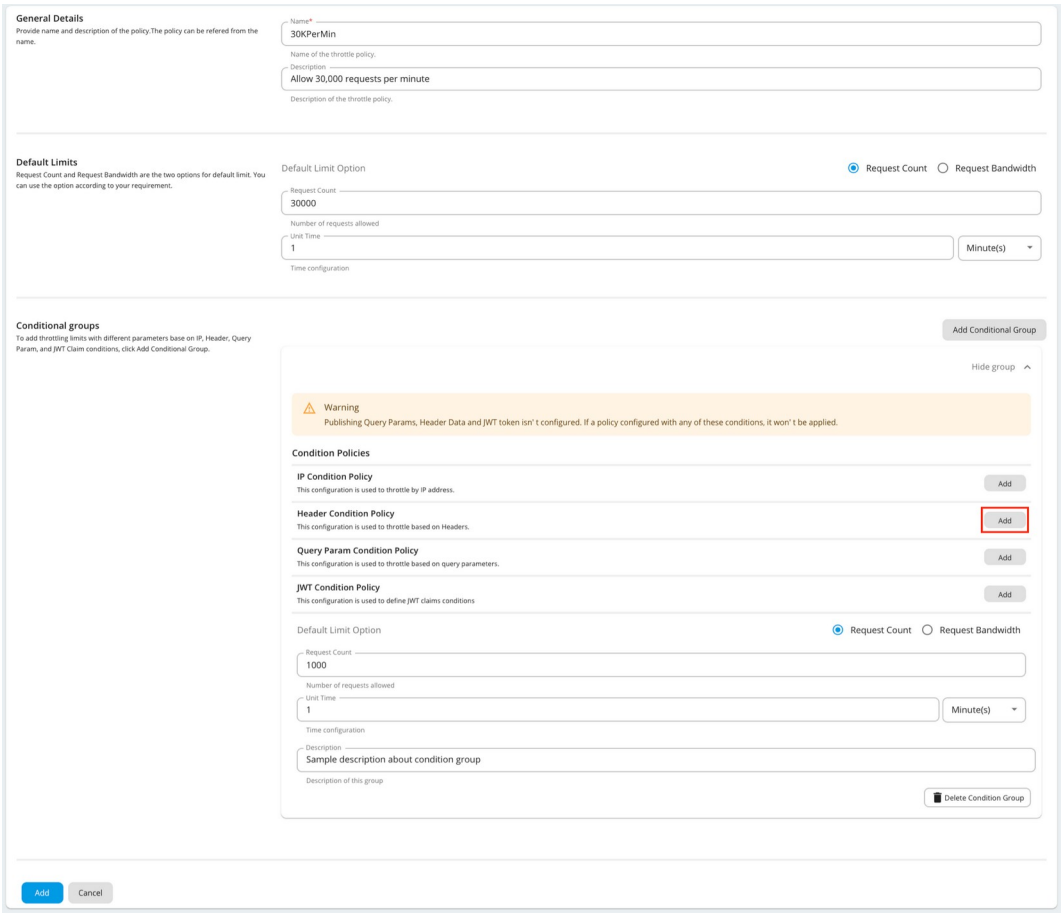
10. Optional – test the application quota: in the Developer Portal, open **PlatinumTestApp**, click **Edit**, set **Shared Quota** to **10PerMin** and save. Wait for the next minute and run the loop from step 8 again.

    About 10 calls return 200, then 429 with code `900803` (application quota), although Platinum allows 25. The lowest limit that applies always wins. Set **Shared Quota** back to **Unlimited** afterwards.

Part 2 – Add Conditions to an Advanced Rate Limiting Policy

11. In the Admin Portal, click **Advanced Policies** and select **Add New Policy**.





CC BY 4.0

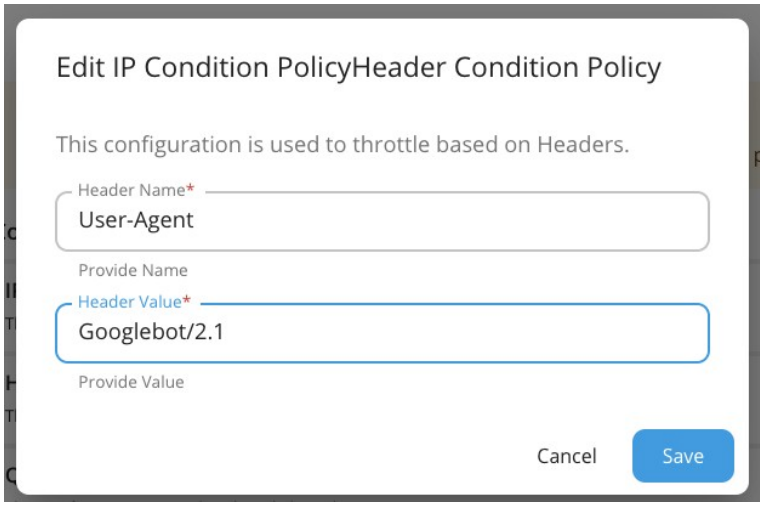
12. Enter the general details of the policy:

Field	Value
Name	`30KPerMin`
Description	Allow 30,000 requests per minute
Default Limits	Request Count: 30000 per 1 Minute(s)

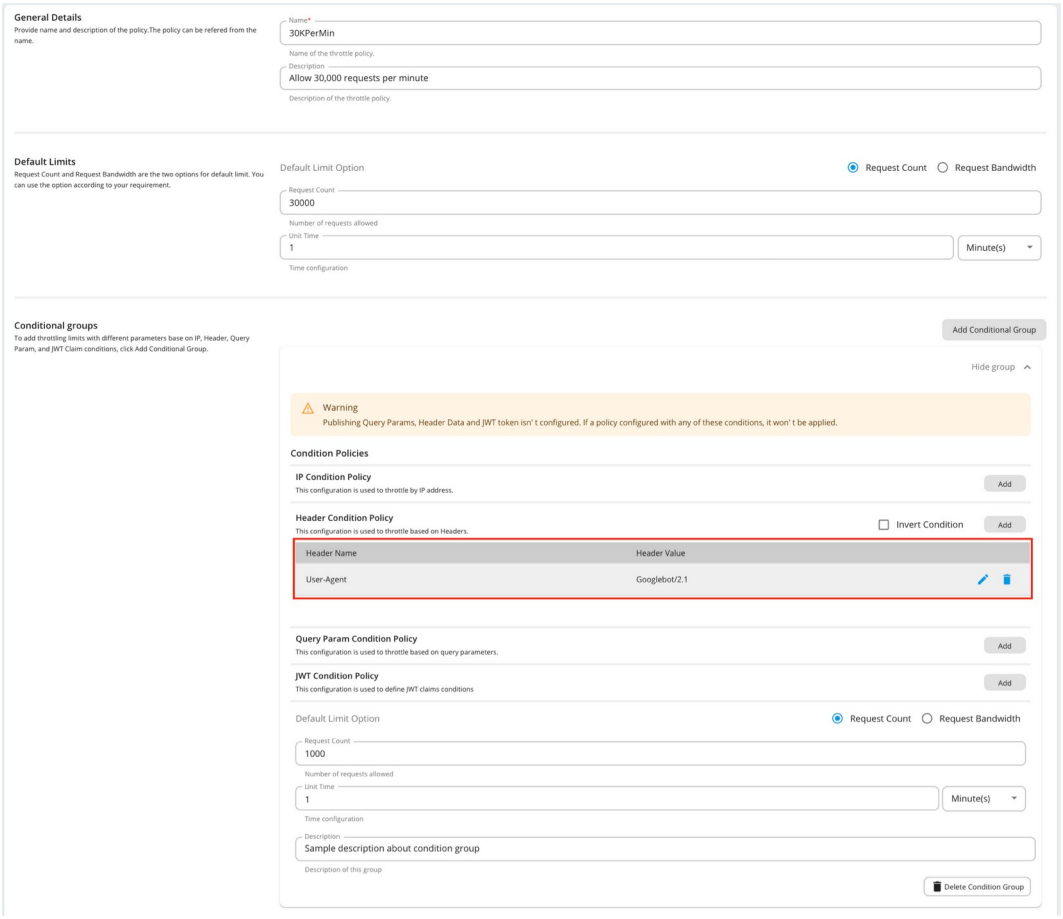
13. Click **Add Conditional Group** and expand the group. Click **Add** next to **Header Condition Policy** and enter the values below. Set the group's limit as shown.

Field	Value
Header Name	`User-Agent`
Header Value	`Googlebot/2.1`
Request Count (group)	1000 per 1 Minute(s)

The conditional group applies throttling to clients that send the User-Agent header with the value Googlebot/2.1, and limits them to 1000 requests per minute.



CC BY 4.0



14. Click **Add** to save the policy.

15. Header, query parameter and JWT conditions only work when they are enabled in the server (the form shows a warning until then). Stop API Manager, open the file below, add the lines that follow, and start API Manager again. If the file already has an `[apim.throttling]` section, add only the second line under it.

```
<APIM_HOME>/repository/conf/deployment.toml
```

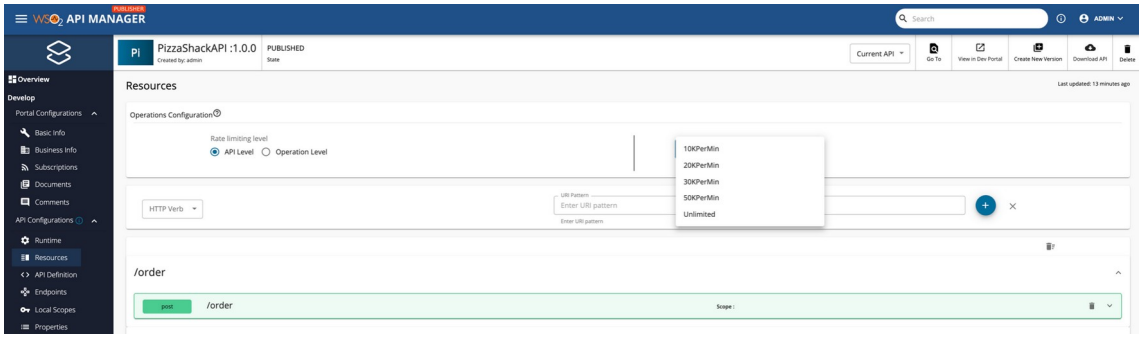
Example:

```
~/wso2am-4.7.0/repository/conf/deployment.toml
```

```
[apim.throttling]
enable_header_based_throttling = true
```

CC BY 4.0

16. The new advanced policy can now be selected for the API in the Publisher: open the API, go to **Resources** and choose **30KPerMin** under **Rate limiting level** (**API Level**, or per operation with **Operation Level**).



Test the advanced policy:

17. To see the Googlebot limit quickly, lower it for the test: in the Admin Portal, open **Advanced Policies**, edit **30KPerMin**, change the conditional group's request count from 1000 to **5** per 1 minute and click **Update**.

18. In the Publisher, make sure **30KPerMin** is selected at **API Level** for the PizzaShack API (step 16). Click **Save** and deploy a new revision. Wait one minute so the earlier Platinum test count is cleared.

19. Send 8 requests that pretend to be Googlebot, using the token from step 7 (generate a new one in **Production Keys** if it has expired):

macOS / Linux

```
for i in $(seq 1 8); do
  curl -k -s -o /dev/null -w "$i %{http_code}\n" \
    -H "User-Agent: Googlebot/2.1" \
    -H "Authorization: Bearer <ACCESS_TOKEN>" \
    "https://<APIM_HOST>:<GW_HTTPS_PORT>/pizzashack/1.0.0/menu"
  sleep 0.5
done
```

Example – macOS / Linux:

```
for i in $(seq 1 8); do
  curl -k -s -o /dev/null -w "$i %{http_code}\n" \
    -H "User-Agent: Googlebot/2.1" \
    -H "Authorization: Bearer eyJ4NXQiOiJNell4.eyJzdWIiOiJhZG1pbiJ9.ZXhhbXBsZQ" \
    "https://localhost:8243/pizzashack/1.0.0/menu"
  sleep 0.5
done
```

Windows (Command Prompt)

```
for /L %i in (1,1,8) do @(curl -k -s -o NUL -w "%i %{http_code}\n" ^
  -H "User-Agent: Googlebot/2.1" ^
  -H "Authorization: Bearer <ACCESS_TOKEN>" ^
  "https://<APIM_HOST>:<GW_HTTPS_PORT>/pizzashack/1.0.0/menu" ^
  & timeout /t 1 /nobreak >NUL)
```

Example – Windows (Command Prompt):

```
for /L %i in (1,1,8) do @(curl -k -s -o NUL -w "%i %{http_code}\n" ^
  -H "User-Agent: Googlebot/2.1" ^
  -H "Authorization: Bearer eyJ4NXQiOiJNell4.eyJzdWIiOiJhZG1pbiJ9.ZXhhbXBsZQ" ^
  "https://localhost:8243/pizzashack/1.0.0/menu" ^
  & timeout /t 1 /nobreak >NUL)
```

CC BY 4.0

About 5 calls return 200 and the rest return 429. Call once more with the header to see the response; code `900800` means the API-level (advanced) policy blocked it (`900802` if the policy is set per operation):

Response (HTTP 429):

```
{"code":"900800","message":"Message throttled out",
 "description":"You have exceeded your quota .You can access API after
  2026-Nov-03 12:12:00+0000 UTC","nextAccessTime":"2026-Nov-03 12:12:00+0000 UTC"}
```

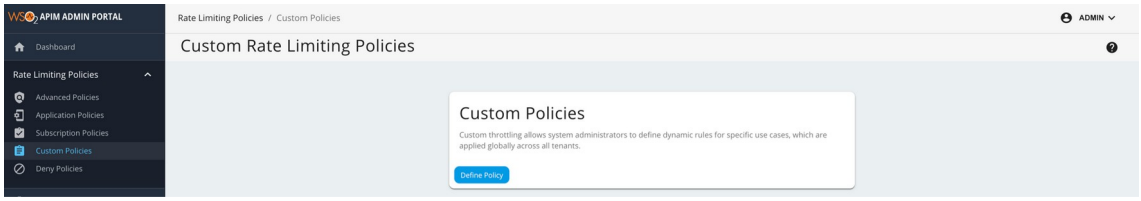
20. In the same minute, run the curl command from step 8 without the User-Agent header. The calls return 200: only requests from Googlebot/2.1 are in the conditional group, and all other clients share the default 30000 per minute.

If the Googlebot calls are never blocked, check that `enable_header_based_throttling` is set and API Manager was restarted (step 15), and that the header value is exactly `Googlebot/2.1`.

21. Set the conditional group's request count back to **1000** per 1 minute and click **Update**.

Part 3 – Add a Custom Rule

22. In the Admin Portal, click **Custom Policies** and select **Define Policy**.



23. Enter the following details, replace the sample in the **Siddhi Query** box with the query below, and click **Add**.

Field	Value
Name	`CustomRule`
Description	Allow 5 requests per minute per user for the PizzaShack API
Key Template	`$userId:$apiContext`

Siddhi Query

```
FROM RequestStream
SELECT userId, ( apiContext == '/pizzashack/1.0.0' ) AS isEligible,
       str:concat(userId, ':', apiContext) AS throttleKey
INSERT INTO EligibilityStream;

FROM EligibilityStream[isEligible==true]#throttler:timeBatch(1 min)
SELECT throttleKey, (count(userId) >= 5) AS isThrottled, expiryTimeStamp
GROUP BY throttleKey
INSERT ALL EVENTS INTO ResultStream;
```

Why not the sample query? The sample in the Admin Portal limits the user 'admin@carbon.super', but in API Manager 4.7 the user ID that the gateway sends with each call is an internal ID (for example 06f9...@carbon.super), not the user name, so the sample never matches anyone. The query above needs no user name: it counts calls per user on the PizzaShack API (apiContext includes the version, /pizzashack/1.0.0). The Key Template must list the same values used to build throttleKey.

CC BY 4.0

24. Test it: wait for the next minute and send 8 requests with the same token, using the loop from step 8 (change 30 to 8):

macOS / Linux

```
for i in $(seq 1 8); do
  curl -k -s -o /dev/null -w "$i %{http_code}\n" \
    -H "Authorization: Bearer <ACCESS_TOKEN>" \
    "https://<APIM_HOST>:<GW_HTTPS_PORT>/pizzashack/1.0.0/menu"
  sleep 0.5
done
```

Example – macOS / Linux:

```
for i in $(seq 1 8); do
  curl -k -s -o /dev/null -w "$i %{http_code}\n" \
    -H "Authorization: Bearer eyJ4NXQiOiJNell4.eyJzdWIiOiJhZG1pbiJ9.ZXhhbXBsZQ" \
    "https://localhost:8243/pizzashack/1.0.0/menu"
  sleep 0.5
done
```

About 5 calls return 200, then 429. Call once more to see the response: code `900806` means a custom policy blocked it. The limit is per user, so another user's token still works in the same minute.

Response (HTTP 429):

```
{"code":"900806","message":"Message throttled out",
 "description":"You have exceeded your quota .You can access API after
  2026-Nov-03 12:13:02+0000 UTC","nextAccessTime":"2026-Nov-03 12:13:02+0000 UTC"}
```

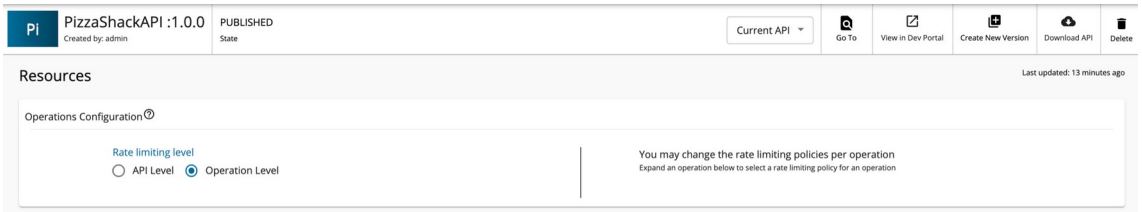
With a token from the client credentials grant (Generate Keys), the user is the owner of the application, for example admin.

25. Delete the **CustomRule** policy (it applies to every user of the PizzaShack API), so that it does not interfere with the next steps.

Part 4 – Block (Deny) an Application

26. Log in to the Publisher and open the **PizzaShack** API.

27. Click the **Resources** tab and make sure **Operation Level** is selected under the operations configuration.



28. Click **Save**, and deploy if you changed the API.

29. Open the Developer Portal, click **Sign-in** and then **Create Account**:

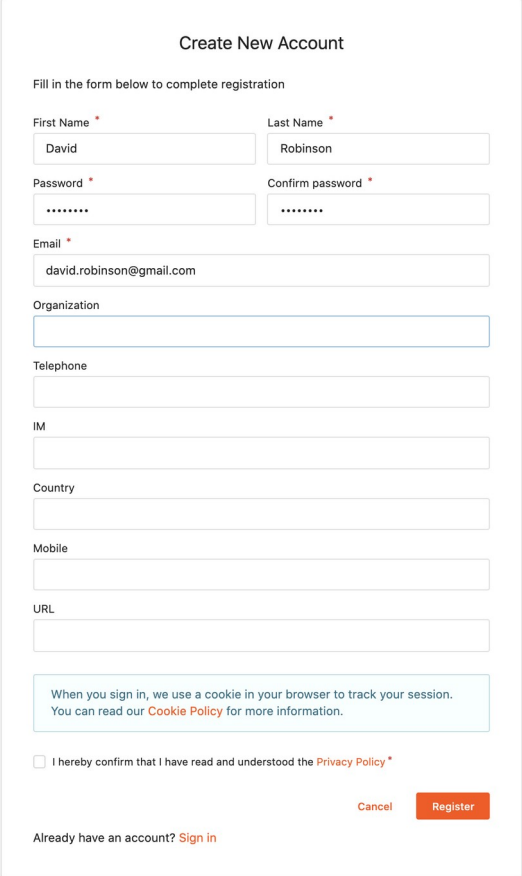
```
https://<APIM_HOST>:<HTTPS_PORT>/devportal
```

Example:

```
https://localhost:9443/devportal
```

CC BY 4.0

30. Self sign up a user named **david** (or use a user you already created).



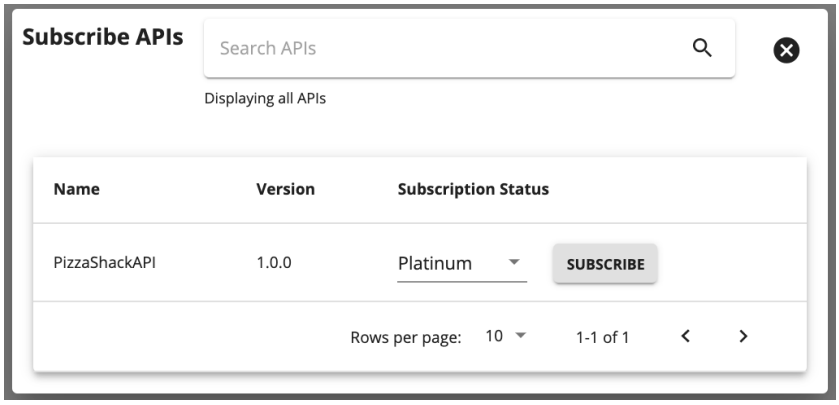
31. Log in to the Developer Portal with the credentials of the user you just created.

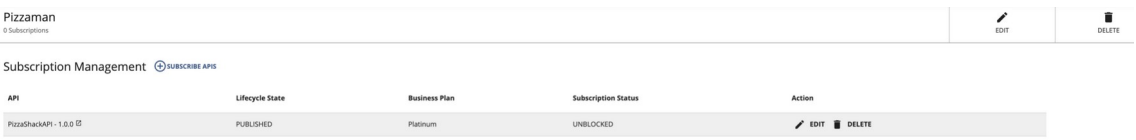
32. If a tenant list is shown, select **carbon.super**.

33. Go to **Applications** and click **Add New Application**.

34. Create an application named **Pizzaman**.

35. Subscribe **Pizzaman** to the PizzaShack API with the **Platinum** business plan.

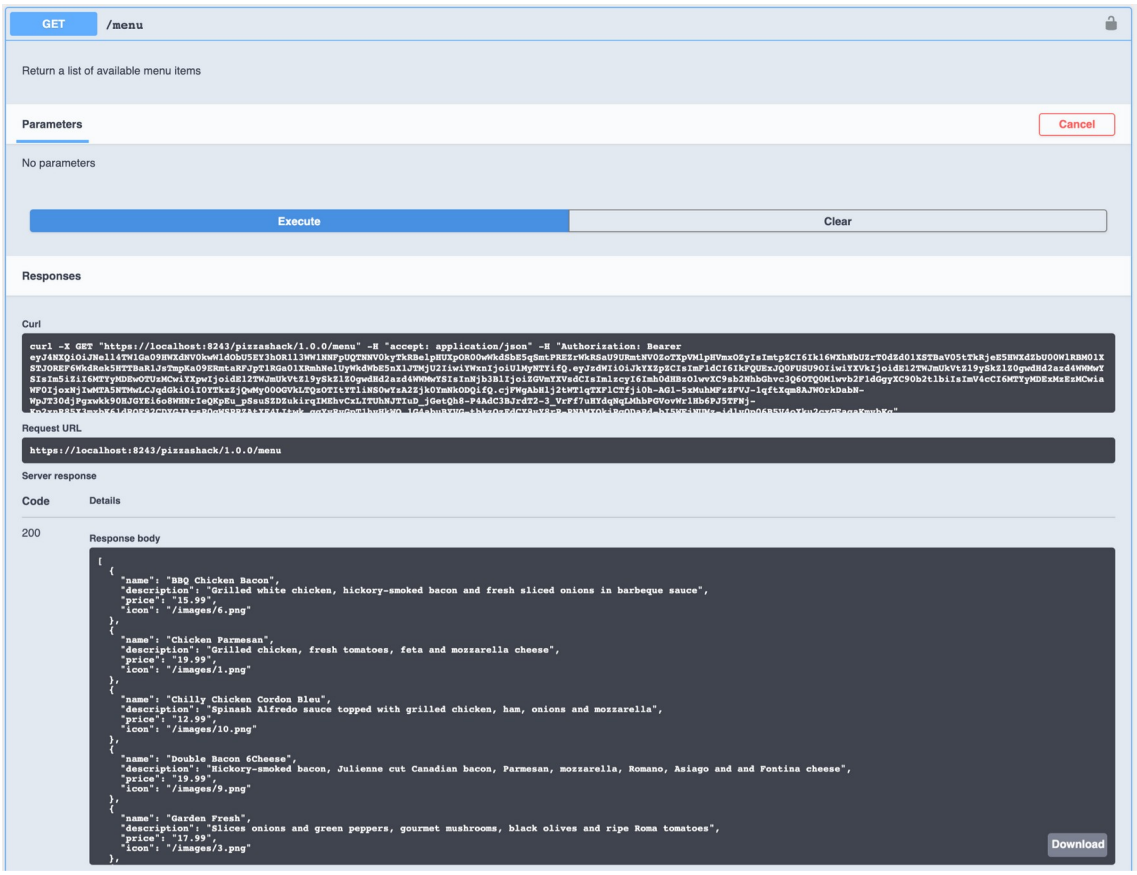




CC BY 4.0

36. Generate OAuth2 keys for the application under the **Production Keys** tab.

37. Go to the **Try Out** console of the PizzaShack API and send a **GET** request to the `/menu` resource. The response is displayed.



Alternatively, use cURL. Replace `<ACCESS_TOKEN>` with the access token generated for the Pizzaman application:

macOS / Linux

```
curl -k -H "Accept: application/json" \
  -H "Authorization: Bearer <ACCESS_TOKEN>" \
  "https://<APIM_HOST>:<GW_HTTPS_PORT>/pizzashack/1.0.0/menu"
```

Example – macOS / Linux:

```
curl -k -H "Accept: application/json" \
  -H "Authorization: Bearer eyJ4NXQiOiJNell4.eyJzdWIiOiJhZG1pbiJ9.ZXhhbXBsZQ" \
  "https://localhost:8243/pizzashack/1.0.0/menu"
```

Windows (Command Prompt)

```
curl -k -H "Accept: application/json" ^
  -H "Authorization: Bearer <ACCESS_TOKEN>" ^
  "https://<APIM_HOST>:<GW_HTTPS_PORT>/pizzashack/1.0.0/menu"
```

Example – Windows (Command Prompt):

```
curl -k -H "Accept: application/json" ^
  -H "Authorization: Bearer eyJ4NXQiOiJNell4.eyJzdWIiOiJhZG1pbiJ9.ZXhhbXBsZQ" ^
  "https://localhost:8243/pizzashack/1.0.0/menu"
```

CC BY 4.0

Or, in a REST client:

```
Method: GET
URL: https://<APIM_HOST>:<GW_HTTPS_PORT>/pizzashack/1.0.0/menu
Authorization: Bearer Token
Token: <ACCESS_TOKEN>
```

Example:

```
Method: GET
URL: https://localhost:8243/pizzashack/1.0.0/menu
Authorization: Bearer Token
Token: eyJ4NXQiOiJNell4.eyJzdWIiOiJhZG1pbiJ9.ZXhhbXBsZQ
```

Pizzaman can now call the API. Like PlatinumTestApp in step 8, it is limited to 25 calls per minute (HTTP **429**, code `900804`).

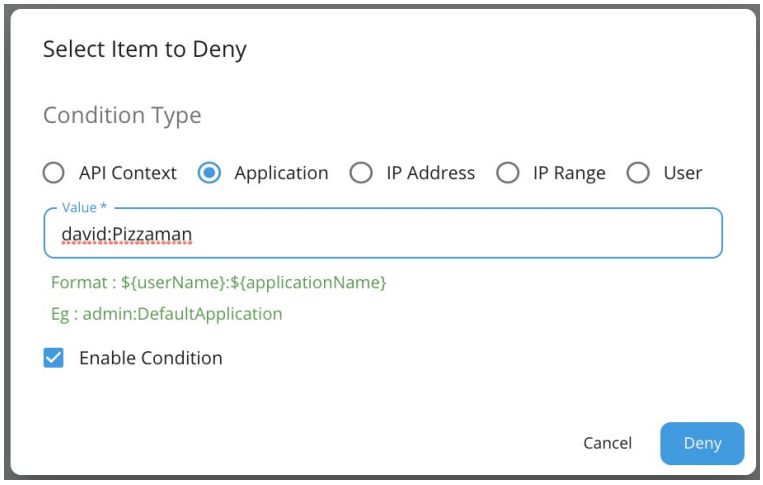
38. Log in to the Admin Portal and click **Deny Policies** under **Rate Limiting Policies**:

```
https://<APIM_HOST>:<HTTPS_PORT>/admin
```

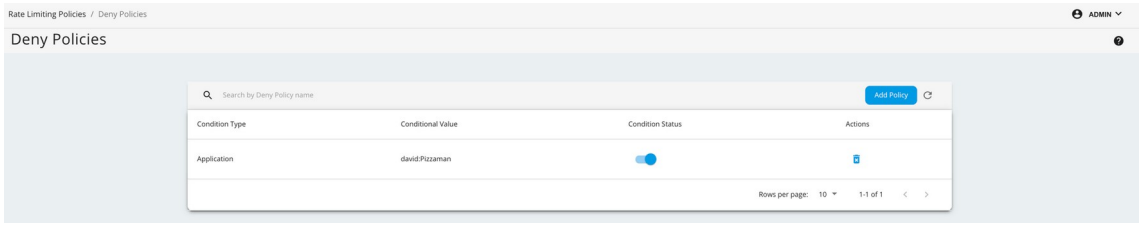
Example:

```
https://localhost:9443/admin
```

39. Click **Add Policy**, select **Application** and enter the value in the format `<username>:<applicationName>`, that is `david:Pizzaman`. Click **Deny**.



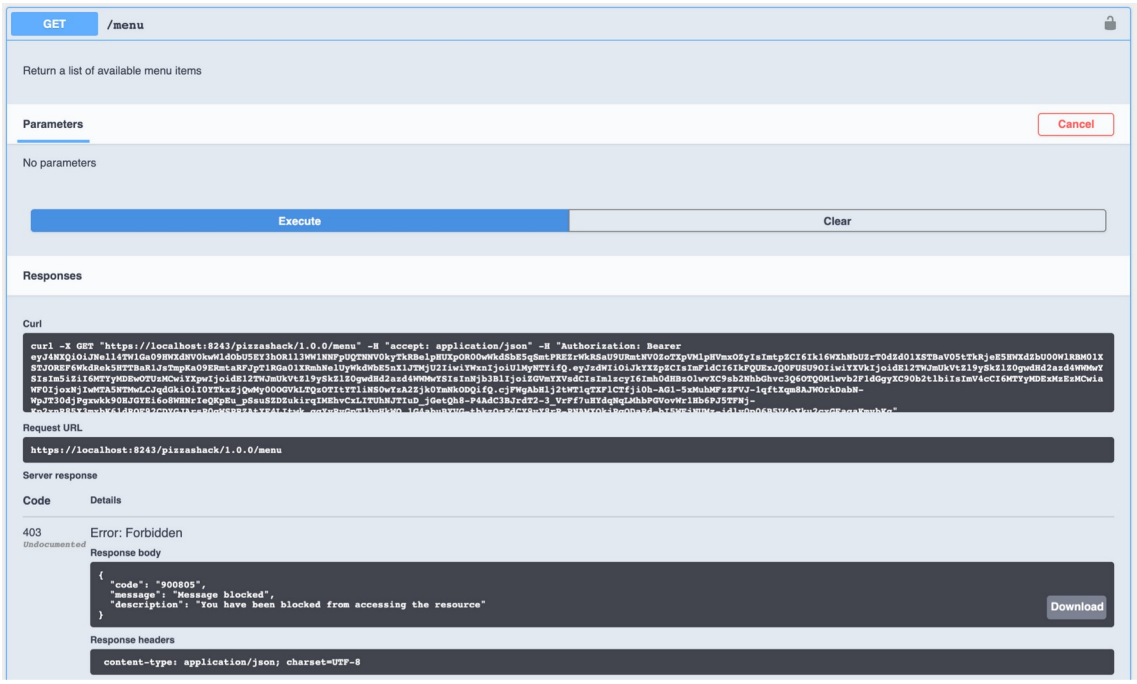
The denied application is listed as follows:



40. Call the PizzaShack API again with the Pizzaman token (Try Out or cURL as above). You receive HTTP 403 with the following response:

```
{"code":"900805","message":"Message blocked","description":"You have been blocked from
accessing the resource"}
```

CC BY 4.0



41. Test that the block is lifted: in the Admin Portal, open **Deny Policies** and delete the `david:Pizzaman` policy. After a few seconds, call the API again with the Pizzaman token. It returns 200 and the menu.

Expected Outcome

The new subscription policy (Platinum) is saved and used by WSO2 API Manager. It is available for selection when you create a new API in the Publisher or edit an existing one, it is shown as a business plan in the Developer Portal, and a subscription on it is limited to 25 calls per minute (HTTP 429, code 900804).

The new advanced rate limiting policy 30KPerMin, with a conditional group, is saved. You can apply it to the whole API or to selected resources. Requests with the User-Agent Googlebot/2.1 get their own limit (HTTP 429, code 900800), while other clients are not affected.

The custom rule limits each user to 5 calls per minute on the PizzaShack API (HTTP 429, code 900806).

Calls to the PizzaShack API by the user david through the Pizzaman application are blocked (HTTP 403, code 900805) while the Pizzaman application is denied, and work again when the deny policy is removed.

CC BY 4.0